A19 Listas

📕 RecyclerView

O RecyclerView facilita e torna eficiente a exibição de grandes conjuntos de dados. Você fornece os dados e define a aparência de cada item, e a biblioteca RecyclerView, quando necessário, cria os elementos dinamicamente [1].

O RecyclerView, como o nome indica, recicla esses elementos individuais. Quando um item rola para fora da tela, o RecyclerView não destrói a visualização dele. Em vez disso, o RecyclerView reutiliza a visualização para novos itens que passaram a aparecer na tela. Isso melhora muito o desempenho, aperfeiçoando a capacidade de resposta do app e reduzindo o consumo de energia [1].

Se você quiser usar o RecyclerView, é necessário realizar algumas ações. Elas serão discutidas em detalhes nas seções a seguir [1].

- Em primeiro lugar, decida como será a lista ou a grade. Em geral, você poderá usar um dos gerenciadores de layout padrão da biblioteca RecyclerView.
- Crie a aparência e o comportamento de cada elemento da lista. Com base nisso, estenda a classe ViewHolder. A versão do ViewHolder fornece toda a funcionalidade para os itens da lista. O fixador de visualização é um wrapper em torno de uma View, e essa visualização é gerenciada por RecyclerView.
- Defina o Adapter que associa seus dados às visualizações ViewHolder.

📙 Planejar seu layout

Os itens na RecyclerView são organizados por uma classe LayoutManager. A biblioteca RecyclerView fornece três gerenciadores de layout, que processam

as situações de layout mais comuns [1]:

- LinearLayoutManager organiza os itens em uma lista unidimensional.
- GridLayoutManager organiza todos os itens em uma grade bidimensional: Se a grade for organizada na vertical, GridLayoutManager tentará fazer com que todos os elementos em cada linha tenham a mesma largura e altura, mas linhas diferentes podem ter alturas diferentes. Se a grade for organizada na horizontal, GridLayoutManager tentará fazer com que todos os elementos em cada coluna tenham a mesma largura e altura, mas colunas diferentes podem ter larguras diferentes.
- StaggeredGridLayoutManager é semelhante a GridLayoutManager, mas não exige que os itens de uma linha tenham a mesma altura (para grades verticais) ou itens na mesma coluna tenham a mesma largura (para grades horizontais). O resultado é que os itens em uma linha ou coluna podem acabar afastados um do outro.

📗 Implementar o adaptador e o fixador de visualização

Depois de determinar seu layout, é necessário implementar o Adapter e o ViewHolder. Essas duas classes trabalham juntas para definir como seus dados são exibidos. O ViewHolder é um wrapper em torno da View que contém o layout de um item individual na lista. O Adapter cria objetos ViewHolder conforme a necessidade e também define os dados para essas visualizações. O processo de associação de visualizações aos dados é chamado de vinculação [1].

Ao definir o adaptador, você precisa modificar três métodos principais [1]:

- onCreateViewHolder(): RecyclerView chama esse método sempre que precisa criar um novo ViewHolder. O método cria e inicializa o ViewHolder e o View associado, mas não preenche o conteúdo da visualização. O ViewHolder ainda não foi vinculado a dados específicos.

- onBindViewHolder(): RecyclerView chama esse método para associar um

ViewHolder aos dados. O método busca os dados apropriados e usa esses dados para preencher o layout do fixador de visualização. Por exemplo, se o RecyclerView exibir uma lista de nomes, o método poderá encontrar o nome apropriado na lista e preencher o widget TextView do fixador de visualização.

- getItemCount(): o RecyclerView chama esse método para ver o tamanho do conjunto de dados. Por exemplo, em um app de lista de endereços, pode ser o número total de endereços. O RecyclerView usa essa função para determinar quando não há mais itens a serem exibidos.

📘 Exemplo

Criação da classe de Adapter

```
class CursoAdapter(var cursos: List<Curso>, var eventos:
CursoAdapter.Evento) : RecyclerView.Adapter<CursoAdapter.ViewHolder>()
{

    // Métodos que devem ser implementados pelo CursoAdapter
    // - onCreateViewHolder: Inicializa o layout
    // - getItemCount: Retorna o número de elementos da coleção
    // - onBindViewHolder: Preenche um item da lista

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ViewHolder {
            val layoutInflater = LayoutInflater.from(parent.context)
        val binding = ItemCursoBinding.inflate(layoutInflater)

        return ViewHolder(binding)
    }

    override fun getItemCount(): Int {
        return cursos.size
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
```

```
        val curso = cursos[position]
        holder.binding.txtCurso.text = curso.titulo
        holder.binding.txtDescricao.text = curso.descricao
        holder.binding.txtValor.text = curso.valor
        holder.binding.btnShare.setOnClickListener { eventos.onShare-
Click(curso) }
        holder.binding.card.setOnClickListener { eventos.onCurso-
Click(curso) }
    }

    // Interface com as respostas de eventos
    interface Evento {
        fun onCursoClick(curso: Curso)
        fun onShareClick(curso: Curso)
    }

    // Classe auxiliar para criar cada um dos itens da lista
    data class ViewHolder(var binding: ItemCursoBinding) : RecyclerVi-
ew.ViewHolder(binding.root)
}
```

Criação do RecyclerView

```
    var cursos: MutableList<Curso> = ArrayList()

    // Inicializa o recycler view com o adaptador
    recyclerView.layoutManager = LinearLayoutManager(context)
    recyclerView.adapter = CursoAdapter(cursos, object : CursoAdap-
ter.Evento {

        override fun onCursoClick(curso: Curso) {
                        // Tratamento do evento
        }

        override fun onShareClick(curso: Curso) {
             // Tratamento do evento
        }
    })
```

📚 Referência bibliográficas

[1] RecyclerView. **Android Developers**. Acessado em 14/01/2021.